

TASK 3 — SQL FOR DATA ANALYSIS

Professional SQL Query Portfolio | TV_Final.csv | 912 records

Objective

Use SQL to extract, filter, aggregate, join and analyze structured TV product data. The queries below cover the required SQL concepts and are written in SQLite-compatible SQL.

Dataset Overview

Metric	Value
Records	912
Unique brands	59
Average selling price	■59,358.61
Average rating	4.23/5
Price range	■4,849 – ■499,990

SQL Queries

Q1. Basic SELECT — view core columns

Select the most relevant fields to inspect the dataset.

```
SELECT Brand, Resolution, Size, selling_price, Rating
FROM tv_data
LIMIT 10;
```

Q2. WHERE — highly rated TVs

Filters TVs with a rating of at least 4.5 and sorts the best-rated models first.

```
SELECT Brand, Resolution, Size, selling_price, Rating
FROM tv_data
WHERE Rating >= 4.5
ORDER BY Rating DESC, selling_price DESC
LIMIT 10;
```

Q3. ORDER BY — highest selling price

Finds the most expensive TVs by selling price.

```
SELECT Brand, Size, selling_price, original_price, Rating
FROM tv_data
ORDER BY selling_price DESC
LIMIT 10;
```

Q4. GROUP BY — brand performance

Summarizes product count, average price and average rating by brand.

```
SELECT Brand,
COUNT(*) AS tv_count,
ROUND(AVG(selling_price), 2) AS avg_selling_price,
ROUND(AVG(Rating), 2) AS avg_rating
FROM tv_data
GROUP BY Brand
ORDER BY avg_rating DESC, tv_count DESC;
```

Q5. GROUP BY — operating-system analysis

Compares TV platforms using count, average price and rating.

```
SELECT operating_system,
COUNT(*) AS tv_count,
ROUND(AVG(selling_price), 2) AS avg_selling_price,
ROUND(AVG(Rating), 2) AS avg_rating
FROM tv_data
GROUP BY operating_system
ORDER BY tv_count DESC;
```

Q6. Aggregate — average revenue proxy

Calculates the average selling price; the dataset has product prices rather than transaction revenue.

```
SELECT ROUND(AVG(selling_price), 2) AS average_selling_price
FROM tv_data;
```

Q7. Subquery — above-average priced TVs

Uses a scalar subquery to identify TVs priced above the overall average.

```
SELECT Brand, Resolution, Size, selling_price, Rating
FROM tv_data
WHERE selling_price > (SELECT AVG(selling_price) FROM tv_data)
ORDER BY selling_price DESC
LIMIT 15;
```

Q8. JOIN — brand category

Demonstrates an INNER JOIN between the product table and brand lookup table.

```
SELECT t.Brand, b.brand_category,
COUNT(*) AS tv_count,
ROUND(AVG(t.selling_price), 2) AS avg_selling_price
FROM tv_data t
INNER JOIN brand_info b ON t.Brand = b.Brand
GROUP BY t.Brand, b.brand_category
ORDER BY avg_selling_price DESC
LIMIT 15;
```

Q9. JOIN — operating-system platform type

Demonstrates a LEFT JOIN while preserving all operating systems in the TV data.

```
SELECT t.operating_system, o.platform_type,
COUNT(*) AS tv_count,
ROUND(AVG(t.Rating), 2) AS avg_rating
FROM tv_data t
LEFT JOIN os_info o ON t.operating_system = o.operating_system
GROUP BY t.operating_system, o.platform_type
ORDER BY tv_count DESC;
```

Q10. View — premium product analysis

Creates a reusable view for TVs priced at or above the overall average, then analyzes it.

```
CREATE VIEW premium_tvs AS
SELECT Brand, Resolution, Size, selling_price, original_price, Rating
FROM tv_data
WHERE selling_price >= (SELECT AVG(selling_price) FROM tv_data);

SELECT Brand, COUNT(*) AS premium_tv_count,
ROUND(AVG(Rating),2) AS avg_rating
FROM premium_tvs
GROUP BY Brand
ORDER BY premium_tv_count DESC, avg_rating DESC;
```

Q11. Calculated field — discount percentage

Calculates the discount percentage from original and selling prices.

```
SELECT Brand, Resolution, Size, selling_price, original_price,  
ROUND((original_price-selling_price)*100.0/original_price, 2) AS discount_pct  
FROM tv_data  
WHERE original_price > 0  
ORDER BY discount_pct DESC  
LIMIT 15;
```

Q12. HAVING — brands with strong rating and enough products

Uses HAVING to filter grouped results after aggregation.

```
SELECT Brand, COUNT(*) AS tv_count, ROUND(AVG(Rating),2) AS avg_rating  
FROM tv_data  
GROUP BY Brand  
HAVING COUNT(*) >= 5 AND AVG(Rating) >= 4.0  
ORDER BY avg_rating DESC, tv_count DESC;
```

Q13. Index optimization

Adds an index on selling_price to support repeated price-range filtering and then runs a filtered query.

```
CREATE INDEX IF NOT EXISTS idx_tv_price ON tv_data(selling_price);  
  
SELECT Brand, Resolution, selling_price  
FROM tv_data  
WHERE selling_price BETWEEN 20000 AND 50000  
ORDER BY selling_price DESC;
```

Interview Readiness

Key concepts covered: SELECT, WHERE, ORDER BY, GROUP BY, HAVING, INNER JOIN, LEFT JOIN, subquery, aggregate functions (COUNT/AVG/MIN/MAX), calculated fields, VIEW and INDEX optimization.